Introduction à l'Intelligence Artificielle

# Panorama de modèles d'IA

**Céline Hudelot & Wassila Ouerdane**
CentraleSupélec
28/09/2026

# Objectifs de ce cours

- Avoir une vue de la diversité des architectures de l'IA
- Comprendre les caractéristiques et les conditions d'utilisation de ces modèles.

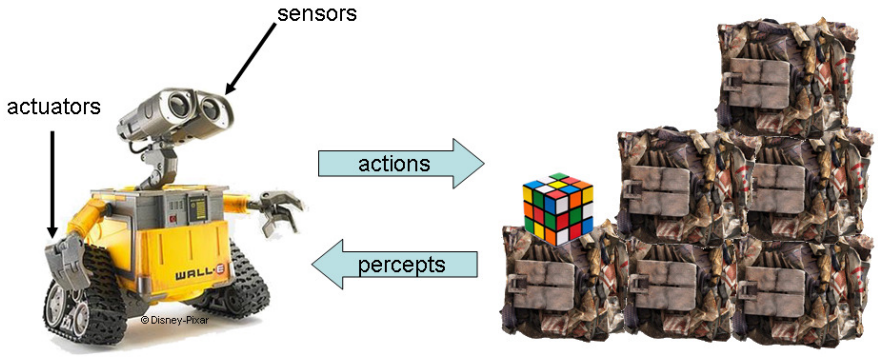
# L’IA vue comme la conception d’agents rationnels

# Reprenons nos notes !

Un agent intelligent doit :

- **Percevoir** et **Agir** dans son environnement
- **Interpréter, Comprendre, Reconnaître** : assimiler et traiter une information
- **Prendre une décision**

# Qu'est ce qu'un agent intelligent ?




Un agent intelligent est une entité qui perçoit son environnement au travers de capteurs et agit sur son environnement au travers d'effecteurs.

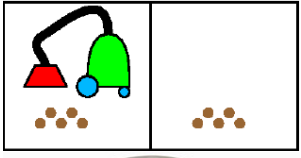
# Définitions

- Perception (percepts) : entrée perçue par l'agent à un moment donné.
- Séquences de perceptions : historique complet de toutes les perceptions.
- Comportement d'un agent = fonction d'agent $f$ qui met en correspondance des séquences perceptives $\mathcal{P}^*$ et des actions $\mathcal{A}$.

$$f : \mathcal{P}^* \to \mathcal{A}.$$

- Implémentation concrète : programme d'agent.
  Il est exécuté sur une architecture concrète pour réaliser (calculer) $f$.

# Exemple classique : l'agent aspirateur



$\mathcal{AS} = (\mathcal{P}_1, \mathcal{A}_1, f)$ où :

- $\mathcal{P}_1 = Loc \times Etat$ avec $Loc = \{A, B\}$ et $Etat = \{propre, sale\}$
  $\mathcal{P}_1$ est un ensemble de couples (Location, Etat de propreté).
- $\mathcal{A}_1 = \{droite, gauche, aspire, rien\}$

# Exemple classique : l'agent aspirateur

| Séquence de perceptions | Action |
|---|---|
| $(A, propre)$ | $droite$ |
| $(A, sale)$ | $aspirer$ |
| $(B, propre)$ | $gauche$ |
| $(B, sale)$ | $aspirer$ |
| $(A, propre)(A, propre)$ | $droite$ |
| $(A, propre)(A, sale)$ | $aspirer$ |
| ⋮ | ⋮ |

Figure – Tabulation partielle de la fonction d'agent aspirateur

# Agent rationnel

Un agent rationnel est celui qui effectue la bonne action, i.e. l'action qui maximise sa mesure de performance en fonction de sa perception du monde et de ses connaissances.

# Agent rationnel

Un agent rationnel est celui qui effectue la bonne action, i.e. l'action qui maximise sa mesure de performance en fonction de sa perception du monde et de ses connaissances.

## Comment mesurer la performance ?

- Avec des critères externes (ex : quantité de poussière aspirée, surface nettoyée, bruit, consommation d'énergie,...)
- Fixée par le concepteur.
- Propre à la tâche.

# Agent rationnel

## Rationnalité

La rationnalité dépend de :

- La mesure de performance (critère de succès).
- La connaissance de l'environnement.
- Les actions possibles de l'agent.
- Les séquences de perceptions de l'agent.

# Environnement de la tâche

## PEAS

La première étape lors de la conception d'un agent est de spécifier l'environnement de la tâche (task environment) qui contient les quatres éléments suivants (PEAS) :

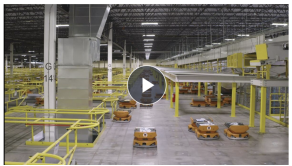
- Mesure de performance (Performance measure).
- Environnement (Environment).
- Effecteurs (Actuators).
- Capteurs (Sensors).

# Exemple : Conducteur de taxi automatique

- Mesure de performance :
  - sécurité, vitesse, légalité, confort, profits,...
- Environnement :
  - routes, autos, piétons,...
- Effecteurs :
  - volant, accélérateur, freins, clignotant,...
- Capteurs :
  - caméras, son, odomètre, indicateur de vitesse, capteur du moteur, ...

# Propriétés de l'environnement [Russel & Norvig 2003]

- Complètement vs partiellement observable – Est ce que les capteurs de l'agent donne un état complet de l'environnement à tout moment?



Partial information→ communication between agents

« These robots follow directions provided by cloud-based algorithms and navigate along a grid of encoded markers. »

*Source : The quest to deploy autonomous robots within Amazon fulfilment centres*

Source : https ://www.amazon.science/latest-news/the-quest-to-deploy-autonomous-robots-within-amazon-fulfillment-centers

# Propriétés de l'environnement [Russel & Norvig 2003]

- Complètement vs partiellement observable – Est ce que les capteurs de l'agent donne un état complet de l'environnement à tout moment?
- Deterministe vs stochastique– L'état de l'environnement dépend t-il de l'état courant et de l'action de l'agent?

*A.-I. Mouaddib, Team MAD, GREYC (Caen)*

Exploration robots
→ learning agents
→ partially observable environment
→ stochastic environment

*Perseverance (NASA) on Mars*

# Propriétés de l'environnement [Russel & Norvig 2003]

- Complètement vs partiellement observable – Est ce que les capteurs de l'agent donne un état complet de l'environnement à tout moment ?
- Deterministe vs stochastique– L'état de l'environnement dépend t-il de l'état courant et de l'action de l'agent ?
- Statique vs dynamique – Est ce que l'environnement peut changer pendant que l'agent délibère ?

Vacuum cleaner robot and Thermostat

# Propriétés de l'environnement [Russel & Norvig 2003]

- Complètement vs partiellement observable – Est ce que les capteurs de l'agent donne un état complet de l'environnement à tout moment?
- Deterministe vs stochastique– L'état de l'environnement dépend t-il de l'état courant et de l'action de l'agent?
- tatique vs dynamique –Est ce que l'environnement peut changer pendant que l'agent délibère?
- Discret vs continu – Est ce que les changements d'états sont discrets (ex :jouer aux échecs) ou continus (ex : se promener)?

continuous environment

# Propriétés de l'environnement [Russel & Norvig 2003]

- Complètement vs partiellement observable – Est ce que les capteurs de l'agent donne un état complet de l'environnement à tout moment?
- Deterministe vs stochastique– L'état de l'environnement dépend t-il de l'état courant et de l'action de l'agent?
- Statique vs dynamique –Est ce que l'environnement peut changer pendant que l'agent délibère?
- Discret vs continu – Est ce que les changements d'états sont discrets (ex :jouer aux échecs) ou continus (ex : se promener)?
- Episodique vs séquentiel– Peut-on évaluer la qualité d'une action à l'intérieur d'un épisode (perception-action)? Ou le développement futur est pertinent pour l'évaluation?

# Conception d'agents

Le type de l'environnement a un impact important sur la conception de l'agent.

- Partiellement observable : l'agent nécessite d'avoir une mémoire.
- Stochastique : l'agent peut à avoir à se préparer à des éventualités.
- Multi-agent : l'agent peut avoir besoin de se comporter de manière aléatoire.
- Statique : l'agent a le temps de prendre une décision rationnelle.
- Temps continu : l'agent doit avoir un fonctionnement continu.
- Physique inconnue : l'agent a besoin d'exploration.
- Mesure de performance inconnue : l'agent a besoin d'interagir et d'observer

# Les différents types d'agent

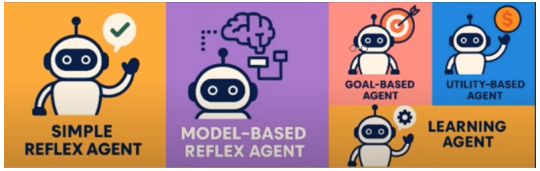



Figure – Source : `https://www.youtube.com/@CodeWithAarohi`

# Les agents reflexes simples

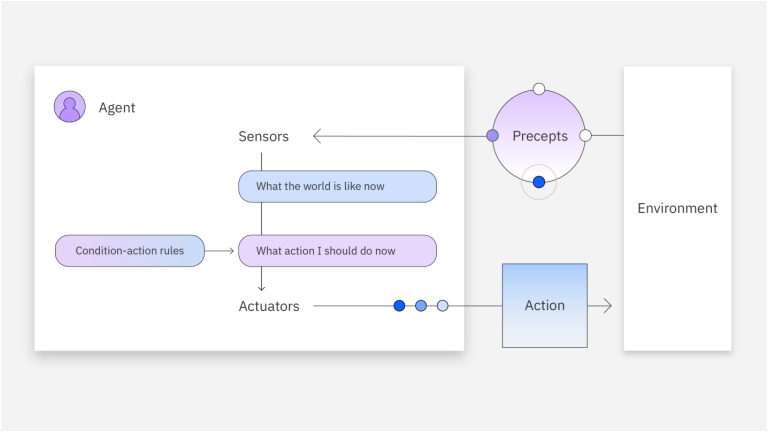



Figure – Source : IBM

Choix de l'action à faire (la décision) en se basant sur la perception courante.

# Les agents reflexes avec modèle

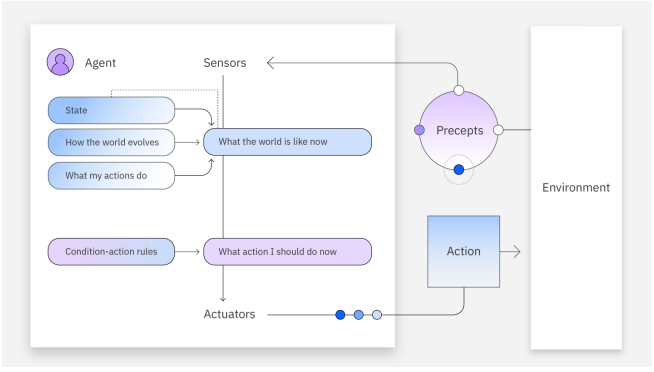



Figure – Source : IBM

Incorpore un **modèle interne du monde** : permet de suivre l'état actuel de l'environnement et à comprendre comment les interactions passées ont pu l'influencer (e.g. : robot qui se déplace dans une pièce qui tient en compte ses mouvements précédents et l'emplacement des obstacles qu'il a déjà franchis.)

# Les agents fondés sur des buts

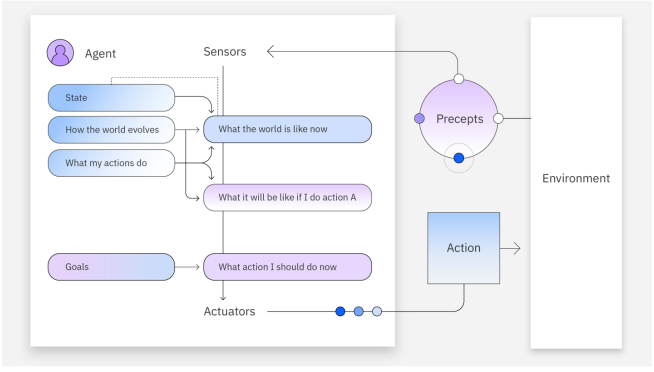



Figure – Source : IBM

Agents qui agissent en se **fixant un objectif spécifique**, qui guide leurs actions. Ils évaluent les différentes actions possibles et choisissent celle qui est la plus susceptible de les aider à atteindre cet objectif, e.g. exploration et planification.

# Les agents fondés sur l'utilité

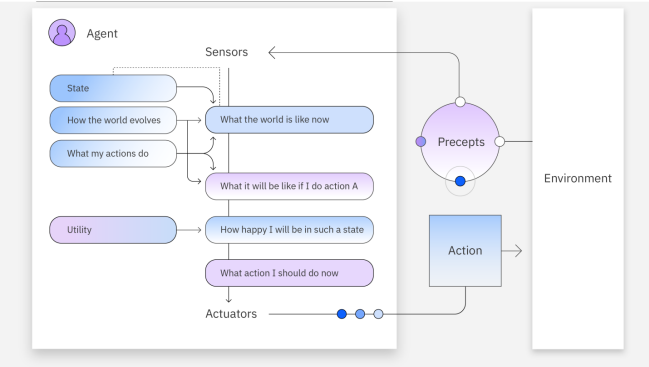



Figure – Source : IBM

Utilisation d'une **fonction d'utilité pour évaluer et sélectionner les actions qui maximisent le bénéfice global**. Les agents basés sur l'utilité considèrent un éventail de résultats possibles et attribuent une valeur d'utilité à chacun d'entre eux, ce qui les aide à déterminer le plan d'action le plus optimal.

# Les agents capables d'apprentissage

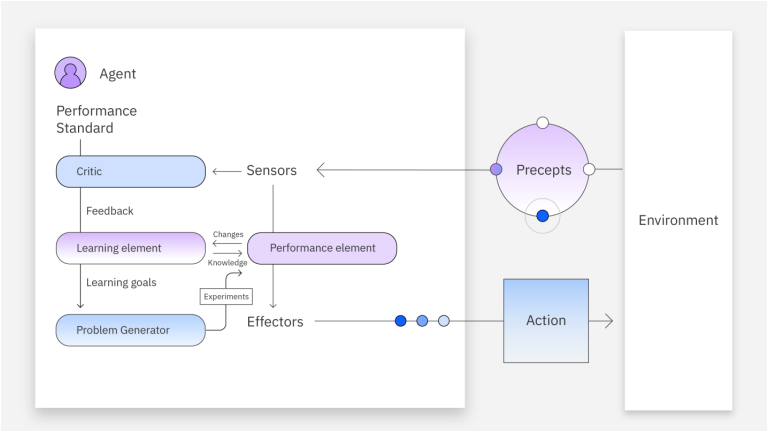



Figure – Source : IBM

Un agent apprenant améliore ses performances au fil du temps en s'adaptant à de nouvelles expériences et données

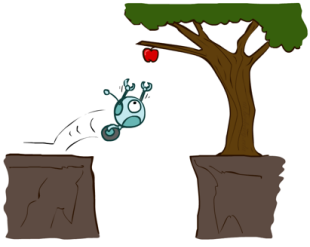
# Les agents capables d'apprentissage

Contrairement aux autres agents, qui s'appuient sur des règles ou des modèles prédéfinis, les agents d'apprentissage actualisent en permanence leur comportement en fonction des informations fournies par l'environnement.
Ils s'appuient sur 4 modules :

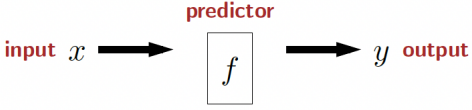
- **(1) module de performance** : prend des décisions basées sur une base de connaissances,
- (**2) module d'apprentissage** : Ajuste et améliore les connaissances de l'agent en fonction du retour d'information et de l'expérience.
- **(3) Critique** : Évalue les actions de l'agent et fournit un retour d'information, souvent sous la forme de récompenses ou de pénalités.
- **(4) Générateur de problèmes** : Suggère des actions exploratoires pour aider l'agent à découvrir de nouvelles stratégies et à améliorer son apprentissage.

# Les agents reflexes

# Les agents reflexes

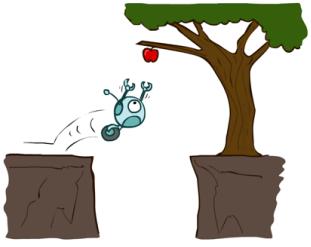


Source : CS188 Intro to AI - Berkeley

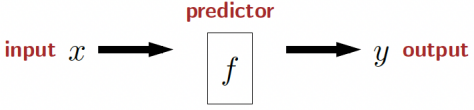



Source : CS221 Intro to AI - Stanford

# Les agents reflexes



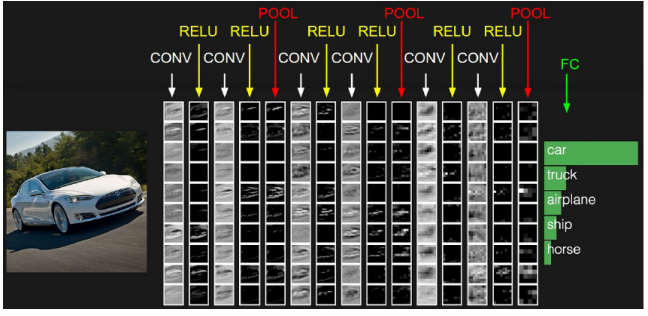
Source : CS188 Intro to AI - Berkeley



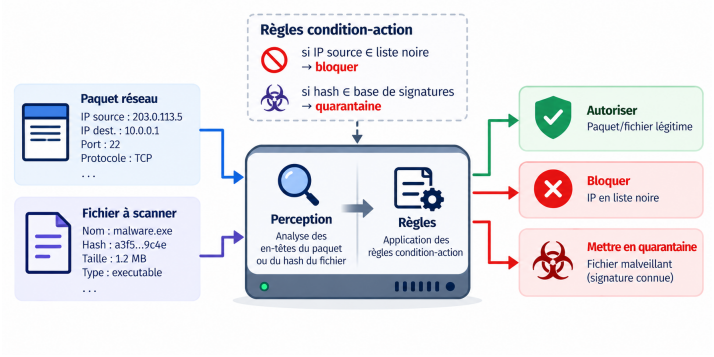

Source : CS221 Intro to AI - Stanford

- Sortie : choix de l'action (une simple action) à faire (la décision) en se basant sur la perception courante.
- Considère le monde comme il est sans prendre en compte les conséquences de ses actions sur le monde.
- **Effectue une séquence déterminée de calculs sur une entrée donnée** (*feedforward*)

# Exemple 1 : les classifieurs



# Exemple 2 : Pare-feu à filtrage de paquets / antivirus à signatures



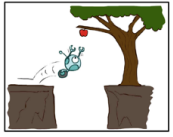

## Limites

Aveugle aux attaques nécessitant du contexte ou de la mémoire : scan de ports lent (un paquet isolé par minute), attaque distribuée dans le temps, malware absent de la base de signatures (zero-day).

# Agent orienté-but - Modèles basés sur des états

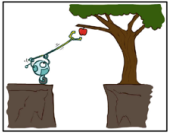
# Modèles à base d'états

**Reflex agent**



- Consider how the world IS

**Planning agent**



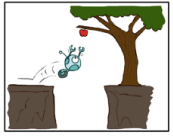
- Consider how the world WOULD BE

## Agent basé sur des buts

- Considère comment le monde pourrait être.

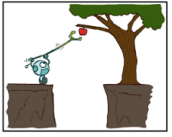
# Modèles à base d'états

**Reflex agent**



- Consider how the world IS

**Planning agent**



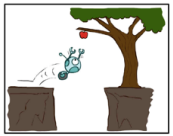
- Consider how the world WOULD BE

## Agent basé sur des buts

- Considère comment le monde pourrait être.
- Les décisions sont basées sur les conséquences hypothétiques des actions.

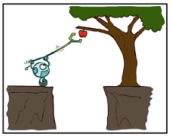
# Modèles à base d'états

**Reflex agent**



- Consider how the world IS

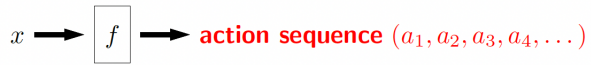
**Planning agent**



- Consider how the world WOULD BE

## Agent basé sur des buts

- Considère comment le monde pourrait être.
- Les décisions sont basées sur les conséquences hypothétiques des actions.
- Un modèle de comment le monde évolue en réponse aux actions est nécessaire = modèles à base d'états et de graphes d'états.



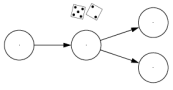
# Modèles à base d'états : les trois grands types

## Problèmes de recherche

Tout est sous contrôle

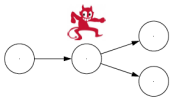
## Processus de Markov

Contre la nature, avec hasard

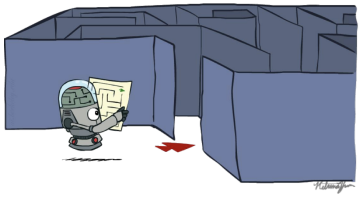


## Recherche adversariale

Contre un adversaire, avec opposition



# Résolution de problèmes par recherche



source : Berkley

# Problème de recherche

## Les principaux ingrédients

1. Spécification du problème :
   Ensemble des états et des actions à considérer pendant la résolution.
2. Spécification du but :
   L'ensemble des états à atteindre.
3. Recherche de solutions :
   Parcourir les différentes séquences d'actions qui amènent aux états solutions souhaités et choisir la meilleure solution.
4. Exécution - Décision :
   Exécution de la séquence d'actions choisie.

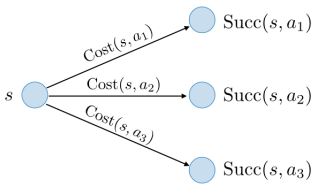
# Résolution de problèmes de recherche

## Principe général

- On représente un problème par un **espace d'états** (arbre - graphe).
- Chaque état est une configuration possible du problème.
- Résoudre le problème consiste à **trouver un chemin dans le graphe**, i.e. déterminer une séquence d'actions $(a_1, a_2, ..., a_n)$ démarrant d'un état initial et aboutissant à un état final.
- Objectif : minimiser le coût associé à ces actions à l'aide de modèles basés sur les états.

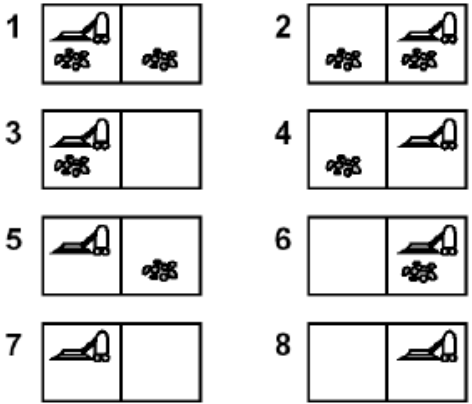
## Hypothèses

- Modèle **déterministe** : en effectuant une action $a$ à partir d'un état $s$, on arrive de manière déterministe à l'état $\mathrm{Succ}(s, a)$.
- L'environnement est totalement observable.
- Modèle discret : états et actions discrets.

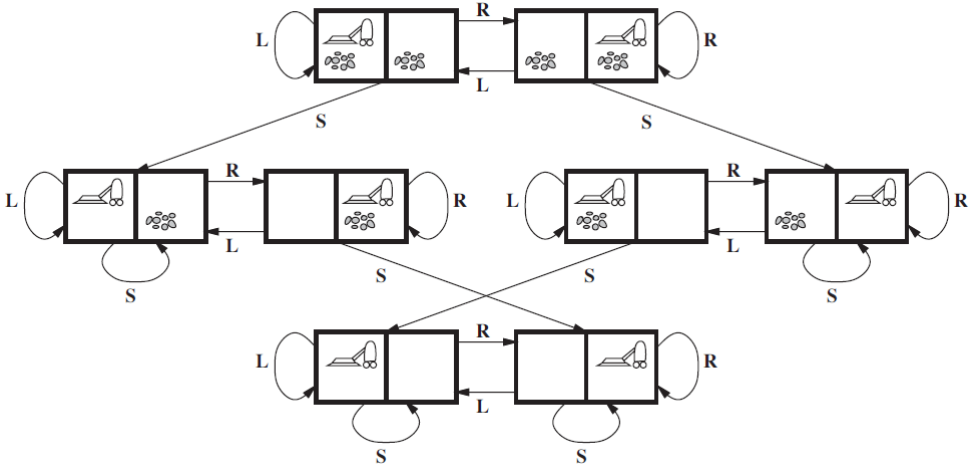


# Exemple : l'agent aspirateur

- Etats : 2 positions + sale ou non : 8 états possibles.
- Etat initial : n'importe lequel
- Function successeur : actions possibles *Gauche, Droite, Aspire*.
- But : éliminer toute la poussière : état 7 ou 8
- Cout : chaque lien vaut 1 donc nombre d'étapes.

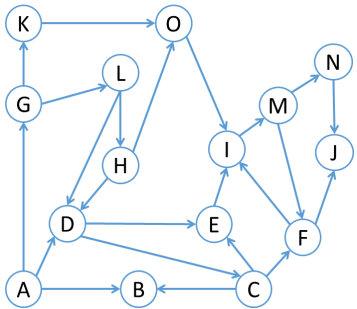


# Exemple : l'agent aspirateur : l'espace d'états

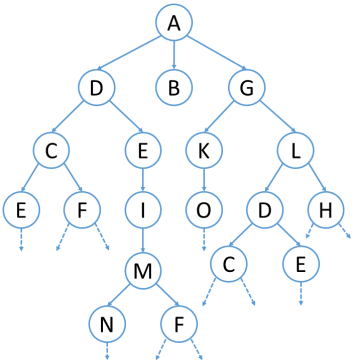


# Représentation par arbre d'un problème

Problème général

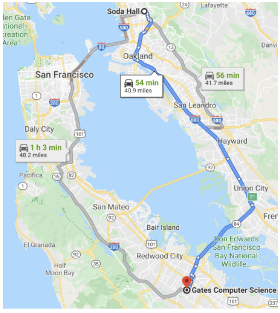



Représentation par arbre



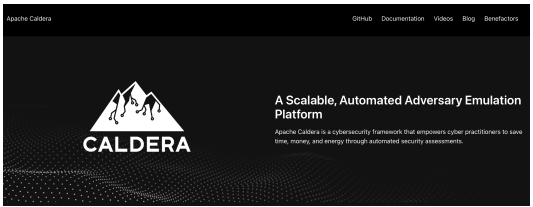
# Exemples de problèmes réels

## Recherche de chemins




- Actions : aller tout droit, tourner à gauche, tourner à droite
- But : Sortie d'une **sequence d'actions** qui permet d'aller de la source à la destination. destination.
- Objectif : plus court ? plus rapide ? plus spectaculaire ?

# MITRE Caldera : émulation d'un adversaire




Une plateforme d'émulation d'adversaire qui automatise l'exécution de scénarios d'attaque à partir de techniques MITRE ATT&CK.

- État : connaissances sur l'environnement (hôtes, comptes, privilèges, …)
- Action : exécution d'une technique ATT&CK
- But : atteindre une condition donnée dans l'environnement

## Une formulation comme problème de recherche

Plusieurs séquences d'actions peuvent être possibles : la recherche consiste à trouver un chemin vers le but dans cet espace d'états.

# Exemples de problèmes réels

## Planification de robots

- Actions : toutes celles possibles pour le robot.
- Objectif : le plus rapide ? le plus économe en énergie ? le plus sûr ? le plus expressif ?

# Recherche dans un arbre

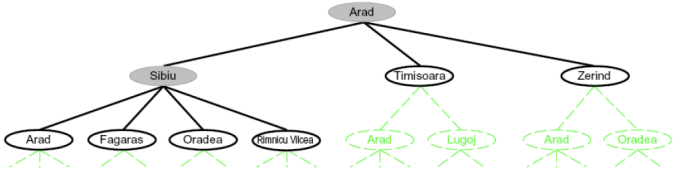



Figure – Source : Berkeley AI course

## Recherche :

- Etendre les plans potentiels (nœuds de l'arbre).
- Maintenir une frontière des plans partiels en considération.
- Etendre le moins de nœuds possibles (principe d'économie).

# Recherche de solution : un algorithme générique

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```

Figure – Source : Berkeley AI course

## Principales idées

- Frontière
- Stratégie d'expansion
- Stratégie d'exploration
- Principale question : quel nœud étendre ?

# Différents types de stratégies

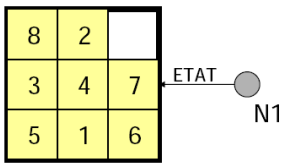
## Stratégies de recherche aveugle

N’exploitent aucune information contenue dans un noeud donné.

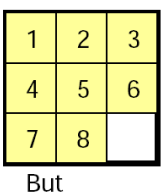
## Stratégies de recherche heuristique

Exploitent certaines informations pour déterminer si un nœud est plus prometteur qu’un autre.

# Exemple : jeu du 8-puzzle

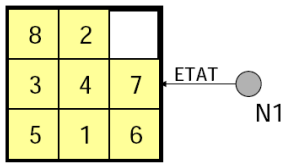



Pour une stratégie aveugle, N1 et N2 sont simplement deux noeuds (à une certaine profondeur de l'arbre de recherche)

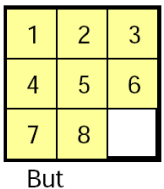



D'après http ://cui.unige.ch/DI/cours/IA

# Exemple : jeu du 8-puzzle




Pour une stratégie heuristique comptant le nombre de plaquettes mal placées, N2 est plus prometteur que N1




D'après http ://cui.unige.ch/DI/cours/IA

# Performances de la stratégie de recherche

## Critères d'évaluation des méthodes de recherche

- Complétude : est ce que la méthode garantit de trouver une solution si elle existe ?
- Optimalité : est ce que la méthode trouve la meilleure solution si il en existe plusieurs ?
- Complexité en temps : temps pris pour trouver la solution ( souvent nombre de noeuds produits)
- Complexité en espace : mémoire nécéssaire (nombre de noeuds à conserver en mémoire pour trouver la solution)
- La complexité est exprimée en utilisant :
  - b : facteur de branchement maximal de l'arbre de recherche : nombre maximum de successeurs à un nœud.
  - s : profondeur de la meilleure solution
  - m : longueur maximale d'un chemin dans l'espace d'etats.

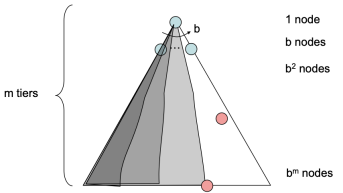
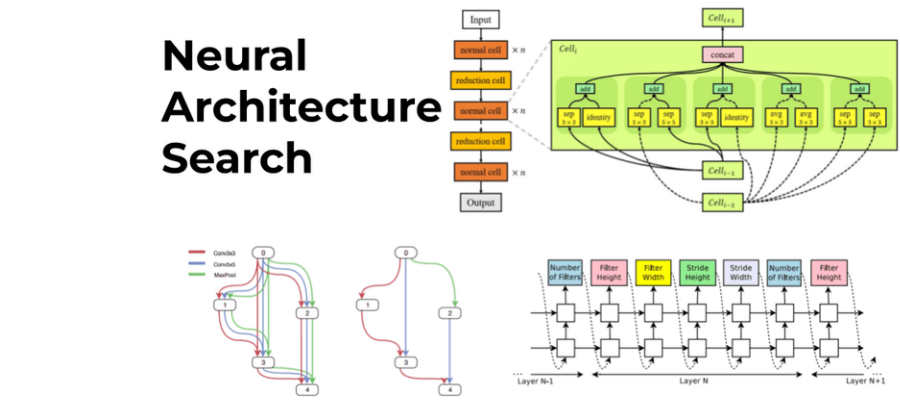


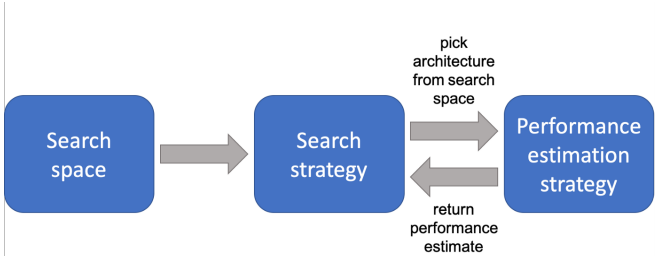

Figure – Source : Berkeley AI course

# Problème de recherche : une application récente




Automatiser le processus d'identification d'architectures efficaces pour un problème d'apprentissage en profondeur donné.

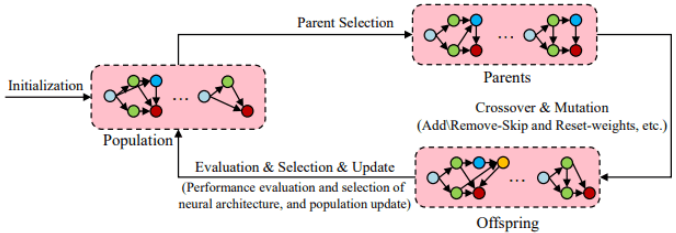
# Recherche d'architecture neuronale




- **Espace de recherche** : L'espace de recherche NAS définit un ensemble d'opérations (par exemple convolution, entièrement connectée, pooling) et la manière dont les opérations peuvent être connectées pour former des architectures de réseau valides
- **Algorithme de recherche** : échantillonne une population de candidats d'architectures de réseau. Il reçoit les métriques de performance du modèle enfant en tant que récompenses (par exemple, haute précision, faible latence) et optimise pour générer des candidats d'architectures de haute performance.
- **Stratégie d'évaluation** : mesurer, estimer ou prédire les performances d'un grand nombre de modèles enfants proposés afin d'obtenir des commentaires pour que l'algorithme de recherche puisse apprendre.

# Recherche d'architecture neuronale

## Avec les algorithmes génétiques




AmoebaNet : `https://arxiv.org/pdf/1802.01548.pdf`

# Recherche d'architecture neuronale

Pour approfondir :

- Revue : Elsken et al, 2029 - Recherche d'architecture neuronale : Une enquête - `https://arxiv.org/abs/1808.05377`
- Blog tutoriel : `https://lilianweng.github.io/posts/2020-08-06-nas/`

# Problème de recherche : Résumé

Les solveurs de problèmes :

- utilisent un graphe pour représenter un problème
- utilisent des heuristiques pour aller plus vite
- ont besoin d'une intervention humaine pour obtenir ce graphe
- utilisent des états symboliques (manque de concision)

# Problème de recherche : résumé

Nous allons maintenant voir deux nouveaux modèles construits selon ce principe mais étendus de deux manières différentes.

- Première extension : on considère que l'agent n'est pas tout seul à agir dans son environnement.
- Deuxième extension : permettre aux états de ne plus être de simples boites noires.

# Problèmes de recherche adversariale

# Problèmes de jeux

## Caractéristiques

- Problème de recherche en présence d'un adversaire (**recherche adversariale**)
  - Introduction d'une incertitude
  - Tous les mouvements ne sont pas contrôlés par l'ordinateur.
- Les programmes de jeux doivent faire face à l'imprévu.
- Complexité : les jeux sont souvent trop complexes pour envisager une solution de recherche exhaustive (exemple : arbre de recherche des échecs)
- Le but de la recherche est de choisir à chaque pas le meilleur prochain coup à jouer

# Jeux traités ici

## Caractéristiques

- Les deux joueurs jouent **tour à tour**
- Les deux joueurs ont **toutes les informations possibles sur la partie** : environnement déterministe et totalement observable
- Les valeurs renvoyées par les fonctions d'utilité des agents à la fin de la partie sont soit nulles soit égales.

# Formulation sous forme d'un problème de recherche

- **Etat** : position des éléments du jeu et identité du joueur qui doit jouer
- **Etat initial** : position initiale des éléments du jeu et identité du joueur qui va commencer.
- **Fonction successeur** : fonction indiquant les coups possibles pour un état donné
- **Etat terminal** : un des joueurs a gagné ou bien il y a égalité.
- **Fonction d'utilité** : renvoie une valeur évaluant les états terminaux : +1 (gagnant), -1 (perdant) et 0 (égalité)

# Définitions

## Arbre de jeu

Arbre qui consiste en des noeuds qui représentent les options des joueurs dans un jeu à deux joueurs. Il y a alternance entre les options des deux joueurs.

## Tour ou Couche

Les différents niveaux dans un arbre de jeu s'appellent les tours ou les couches

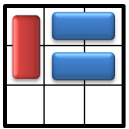
## Noeuds finaux

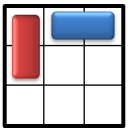
Ils indiquent la fin du jeu : soit une configuration gagnante, perdante ou un match nul

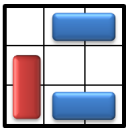
# Les dominos de couleurs

## Principe

Sur une grille nxn, il faut être le dernier à poser son domino 2x1. Le joueur 1 joue horizontalement, l'autre verticalement

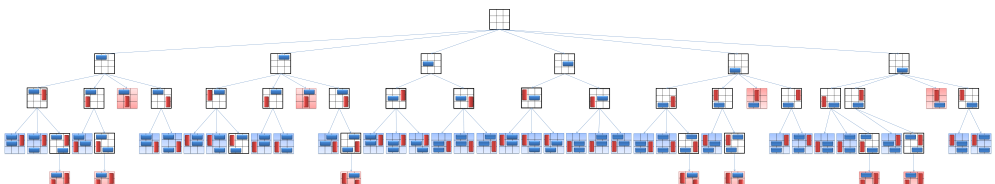




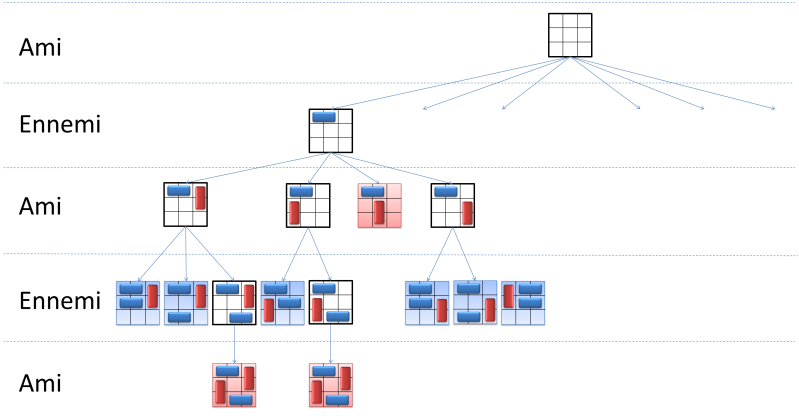


# Exploration de l’arbre de jeu

- La simulation de tous les coups possibles à partir de l’état initial permet de construire l’arbre de jeu.



# Exploration de l’arbre de jeu

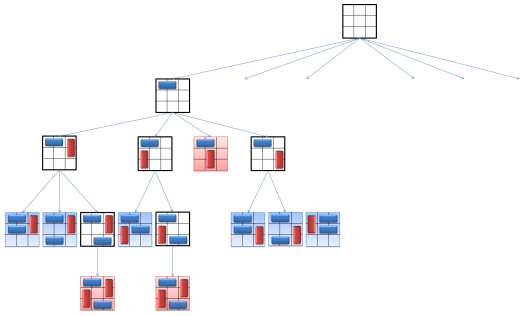



## Exploration de l’arbre de jeu

Parcours en profondeur d’abord (Depth-First Search, DFS) dans un arbre décrit en intention (non construit réellement en mémoire)

# Exploration de l’arbre de jeu

## Exploration complète de l’arbre de jeu



## Pour une grille de n

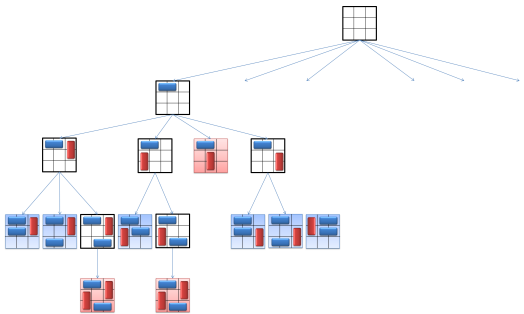
Explosion combinatoire dès que l’on dépasse une grille de 6. Comment améliorer la recherche de solutions ?

# Exploration de l'arbre de jeux : améliorations

## Ne pas ré-explorer les sous graphes communs

- Idée : Dans certains jeux, un même état peut être atteint par différents endroits. Idée : on ne réexplore pas les sous-arbres déja vus.
- On peut aller plus loin en supprimant tous les états symétriques

## Cas des dominos

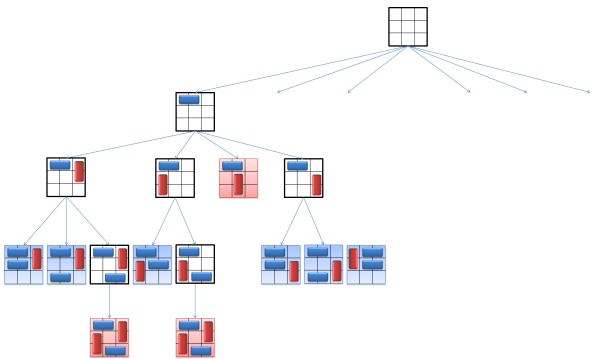


# Exploration de l'arbre de jeux : améliorations

## Coupe intelligente

- Idée : Si une branche amie mène à la victoire, ne pas développer les branches voisines
- Si une branche ennemie mène à la défaite, ne pas développer les branches voisines

## Cas des dominos



# Exploration de l'arbre de jeux : améliorations

## Coupe intelligente

- Trouve si il existe une stratégie gagnante
- Permet de couper (élagage) des parties entières de l'arbre
- mais oblige à aller jusqu'aux feuilles de l'arbre (ce qui est souvent irréalisable en pratique)

# Introduction d'heuristiques pour une recherche avec horizon

## Idée

- On ne développe l'arbre de recherche que jusqu'à une certaine valeur maximale p. Les feuilles de l'arbre ne sont plus forcement les positions finales du jeu.
- On doit donc évaluer les positions. L'information *perdu* ou *gagné* n'est plus suffisante

## Définition

Les heuristiques de jeux sont des fonctions qui associent un réel aux plateaux de jeux. Dans les jeux avec adversaire, plus l'heuristique est grande plus on est proche de la victoire et inversement plus elle est négative et faible et plus on est proche de la défaite.

# Cas des dominos

## Exemple

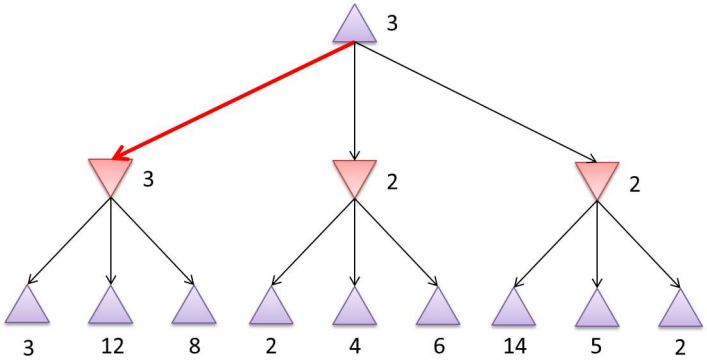
- On développe l'arbre de recherche jusqu'à une profondeur donnée
- On évalue la position aux feuilles
- On fait remonter l'évaluation pour trouver le meilleur coup : algorithme min-max

# Algorithme MiniMax

## Principe

- On s'intéresse à un des deux joueurs qu'on appelle Max. Les valeurs d'utilité sont exprimées du point de vue de Max.
- Le but est de faire gagner Max : trouver la séquence d'actions qui permet d'arriver à un état terminal dont l'utilité est élevée ?
- **NON** : Max doit tenir compte des actions de Min.
- Le principe de l'algorithme Minimax est de développer complètement l'arbre de jeux, de noter chaque feuille avec sa valeur et de faire remonter ces valeurs avec l'hypothèse que chaque joueur choisit le meilleur coup pour lui :
  - valeur minimum aux noeuds MIN
  - valeur maximum aux noeuds MAX

# Algorithme MiniMax : principe




## Principe

- A l'état initial Max a le choix entre trois actions.
- Min a alors le choix entre trois actions dépendantes de l'action de Max au tour précédent.

# Exemple : jeu adversarial en cyberdéfense – Stackelberg

Modéliser la cyberdéfense comme un jeu à deux joueurs : le défenseur s'engage d'abord sur une stratégie de protection, puis l'attaquant observe cette stratégie et choisit sa meilleure réponse.

- Défenseur (leader) : alloue ses ressources limitées (patchs, capteurs, surveillance, protection des cibles).
- Attaquant (suiveur) : observe la stratégie du défenseur et choisit une cible ou une action d'attaque.
- Particularité : les décisions sont séquentielles : le défenseur doit anticiper la réponse de l'attaquant.

## Applications

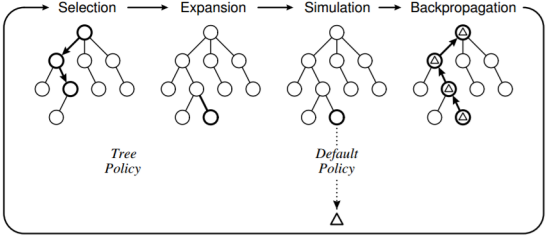
Les Stackelberg Security Games ont été utilisés pour l'allocation de ressources de sécurité physique et ont également été étudiés pour la protection des infrastructures et réseaux informatiques.

# Monte-Carlo Tree Search

## Utilisation

Quand l'espace de recherche est **trop grand ou que la fonction d'évaluation est complexe** (par ex. : le jeu de Go), on utilise une méthode dite de Monte-Carlo.

## Principe



# Monte-Carlo Tree Search

## 4 étapes

- Sélection : à partir de la racine, un choix est fait récursivement pour descendre jusqu'à un nœud qu'il est urgent d'étendre parmi les nœuds extensibles (i.e. pas terminaux et qui possèdent au moins un fils non visité).
- Expansion : un ou plusieurs fils sont ajoutés au nœud sélectionné précédemment en fonction des actions disponibles.
- Simulation : simulations de parties pour produire un score.
- Propagation arrière : on met à jour les statistiques des nœuds de la branche parcourue.

Chaque nœud mémorise :

- le nombre de fois qu'il a été visité
- le score obtenu

# Monte-Carlo Tree Search

## Stratégies pour choisir le prochain coup à jouer

On choisit une des actions à partir de la racine selon un critère parmi :

- Max : on choisit l'action qui mène au nœud qui a la récompense la plus grande ;
- Robustesse : on choisit l'action qui mène au nœud le plus visité ;
- Max et robustesse : on choisit l'action qui mène au nœud qui a à la fois la plus grande récompense et le plus visité. Si un tel nœud n'est pas trouvé, on essaie au moins d'atteindre un nombre de visites acceptables et on choisit le nœud qui a la récompense la plus grande.
- ...

# Méthodes de Monte-Carlo

## Comment choisir le nœud à étendre ?

Dilemme dit d'exploration/exploitation :

- faire plus de simulations pour des coups qui semblent les meilleurs (pour diminuer l'incertitude de leur évaluation),
- mais on veut explorer les moins bons dans le cas où ils seraient en fait meilleurs.

# Méthodes de Monte-Carlo

## Critère UCB (Upper Confidence Bound)

Sélectionner l'action qui maximise le critère UCB :

$$\mu_i + C \times \sqrt{\log(p) \div p_i}$$

où

- $\mu_i$ est la moyenne des scores des parties commençant par ce coup,
- $p$ est le nombre de parties jouées
- $p_i$ est le nombre de parties commençant par le coup
- $C$ est une constante qu'il faut régler pour adapter l'algorithme au problème.

## Comment choisir $C$ en pratique ?

$C$ dépend du jeu.
Une constante élevée favorisera l'exploration alors qu'une petite constante favorisera l'exploitation.

# Alpha GO

Although progress has been steady,
it will take many decades of research and development
before world-championship calibre GO programs exist.

Jonathan Schaeffer, 2001

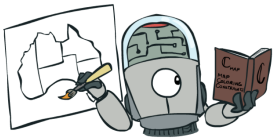
Alpha GO, 2016, bat un joueur de niveau mondial.

# Alpha GO

## Principe

- basé sur MCTS
- utilisation de réseaux de neurones pour choisir le nœuds à étendre, pour simuler les parties et pour évaluer un plateau de jeu
- utilise à la fois des parties jouées par des humains mais également des parties jouées contre lui-même

# Modèles basés sur les variables - Satisfaction de contraintes



Source : Berkley

# Petit rappel

- Dans les modèles précédents, nous avons vu que certains problèmes peuvent être résolus par une recherche dans un espace d'états.
- Ces états sont évalués par des heuristiques propres aux domaines et comparés avec des états buts.
- Dans une telle approche, les états sont **des boites noires** : on ne s'intéresse pas à la structure d'un état.

# Petit rappel

- Dans les modèles précédents, nous avons vu que certains problèmes peuvent être résolus par une recherche dans un espace d'états.
- Ces états sont évalués par des heuristiques propres aux domaines et comparés avec des états buts.
- Dans une telle approche, les états sont **des boites noires** : on ne s'intéresse pas à la structure d'un état.

## Idée

- Identifier parmi les champs décrivant un état, ceux qui varient et contraindre ces variations pour faciliter la recherche.
- Représentation explicite des **variables** et de leurs **contraintes** et des algorithmes de manipulation de contraintes.

# Problèmes à satisfaction de contraintes (CSP)

## Définition

Un problème à satisfaction de contraintes est défini par :

- Un ensemble de variables $\{X_1, X_2, ..., X_n\}$.
  - Chaque variable a un domaine $D_i$ de valeurs possibles (souvent discret et fini).
- Un ensemble de contraintes $\{C_1, C_2, ..., C_p\}$.
  - Chaque contrainte concerne un sous ensemble de variables et spécifie les combinaisons de valeurs autorisées pour ce sous ensemble.

# Problèmes à satisfaction de contraintes (CSP)

Les contraintes peuvent être définies :

- par extension : pour une contrainte donnée, on donne toutes les valeurs autorisées des variables
- par intension : les valeurs sont déduites de la relation exposée par la contrainte Par exemple, pour deux variables $x$ et $y$ définies sur les booléens, la contrainte $x \neq y$

## Exemple

Soit deux variables $x$ et $y$ booléennes. On souhaite représenter la contrainte $c =$ "$x$ est différente de $y$".

- Par extension : $c = \{(x = vrai, y = false), (x = false, y = vrai)\}$
- Par intension : $c = x \neq y$

# Problèmes à satisfaction de contraintes (CSP)

Un état du problème est défini par une affectation de valeurs pour un sous ensemble des variables.

- Si aucune contrainte n'est violée, l'état est consistant.
- Si toutes les variables sont concernées, l'état est complet.
- Si l'état est consistant et complet, c'est une solution.

## Objectif

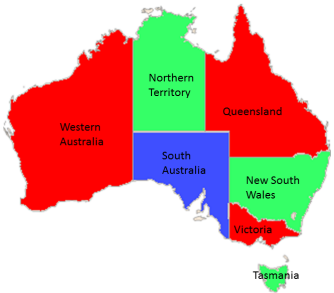
Affecter une valeur à toutes les variables de sorte que toutes les contraintes soient satisfaites.

# Exemple 1 : Problème de coloration de carte

## Description du problème

- **Objectif** : Colorier la carte.
- **Variables** : $\{WA, NT, SA, Q, NSW, V, T\}$
- **Domaines** : {Rouge, Vert, Bleu }
- **Contraintes** : Les régions adjacentes doivent avoir des couleurs différentes.
  $\{WA \neq NT, WA \neq SA, NT \neq Q, NT \neq SA, Q \neq SA, Q \neq NSW, SA \neq NSW, SA \neq V, NSW \neq V\}$

# Exemple 1 : Problème de coloration de carte




Les solutions sont des affectations qui satisfont toutes les contraintes, soient :
{ *WA = rouge, NT = vert, SA = bleu, Q =rouge, NSW = vert, V= rouge, T =vert* }

# Exemple 2 : vérification de pare-feu

## Contexte

Dans Microsoft Azure, des pare-feu protègent les machines des datacenters. Lors d'une modification de leur configuration, il faut vérifier que les nouvelles règles autorisent et bloquent les mêmes communications.

- **Variables** : champs d'un paquet (adresse source, destination, port, protocole)
- **Domaines** : valeurs possibles de ces champs (adresses IP, ports, protocoles)
- **Contraintes** : règles d'autorisation et de blocage des deux pare-feu
- **Question** : existe-t-il un paquet accepté par l'un des pare-feu et rejeté par l'autre ?

### Avec Z3

Le problème est traduit en contraintes logiques et soumis au solveur SMT (Satisfiability Modulo Theories) **Z3**. Il cherche un contre-exemple : un paquet qui révèle une différence entre les deux configurations. ⇝ Si aucun contre-exemple n'existe, les deux configurations sont équivalentes.

Source : A. Helwer, "Checking Firewall Equivalence with Z3", 2018

# Exemple3 : Contrôle d'accès – séparation des tâches

## Contexte

Attribuer des rôles aux utilisateurs tout en respectant des contraintes de *séparation des tâches* (*Separation of Duty*).

- **Variables** : utilisateurs $\{U_1, U_2, \ldots, U_n\}$
- **Domaines** : rôles possibles (*Initiateur, Approbateur, Auditeur, Admin*)
- **Contraintes** :
  - un utilisateur ne peut pas être à la fois *Initiateur* et *Approbateur* d'une même transaction ;
  - un *Auditeur* ne peut pas être *Admin* du système audité ;
  - certaines fonctions doivent être réparties entre plusieurs utilisateurs distincts.

### Formulation CSP

Chercher une affectation des rôles aux utilisateurs qui satisfait l'ensemble des contraintes de sécurité.

*Source : Workflow Satisfiability Problem (WSP)*

# Types de problèmes à satisfaction de contraintes

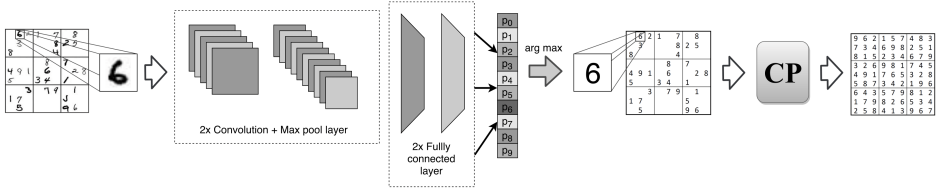
- Domaines **finis/infinis**
  - Domaines finis : nombre de variables n et de grandeur d (CSP booléens)
  - Domaines infinis : entiers, chaînes de caractères (Ordonnancement de tâches : les variables sont les jours de début et de fin de tâches)
- Variables **discrètes /continues**
- Contraintes **linéaires / non-linéaires**
- Cardinalité des contraintes :
  - Unaire : ne concerne qu'une seule variable (*ex : SA =vert*)
  - Binaire : concerne des paires de variables (*ex : SA* $\neq$ *WA*)
  - Ordre supérieur : concerne 3 variables ou plus
- Contraintes **absolues / relatives** : préférences sur les contraintes (exemple : rouge est mieux que vert).
- Contraintes **globales** : contraintes portant sur l'ensemble des variables (exemple : AllDiff).

# Types de problèmes à satisfaction de contraintes

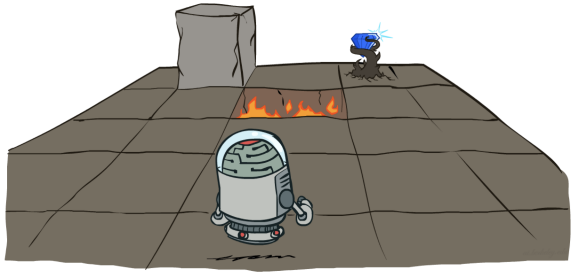
- Satisfaction booléenne (SAT) :
  les variables sont des booléens, les facteurs sont des formules logiques : SAT est NP-complet, mais des progrès extraordinaires ont été réalisés dans les solvers SAT
- Programmation linéaire (LP) :
  les variables sont des réels, les facteurs sont des inégalités linéaires : peuvent être résolus efficacement à l'aide de méthodes spécialisées (par exemple, l'algorithme du simplexe)
- Programmation linéaire en nombres entiers (ILP) :
  Les variables sont des entiers, les facteurs sont des inégalités linéaires : difficiles à résoudre en général
- Programmation mixte en nombres entiers (MIP) :
  les variables sont des réels et des entiers, les facteurs sont des inégalités linéaires : difficiles à résoudre en général

# CSP : hybridation avec le ML

- Visual sudoku assistant : `https://ai.kuleuven.be/stories/post/2023-02-08-sudoku/`
- Test it on Android



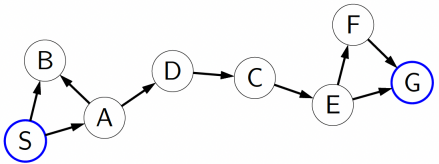
# Processus de décision de Markov



Source : Berkley

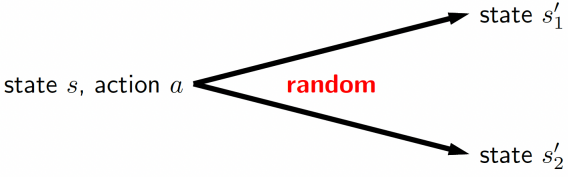
# Jusqu'à présent : modèles basés sur les états

- Nous avons examiné les **problèmes de recherche**, un paradigme puissant qui peut être utilisé pour résoudre une gamme diverse de problèmes.
- Ils supposent que prendre une action $a$ à partir d'un état $s$ résulte de manière **déterministe** en un unique état successeur $\mathrm{Succ}(s; a)$.



# Incertitude dans le monde réel

Dans le monde réel, l'hypothèse du successeur déterministe est souvent irréaliste : prendre une action peut conduire à l'un des nombreux états possibles.



# Processus de décision de Markov

## Jeu de dés

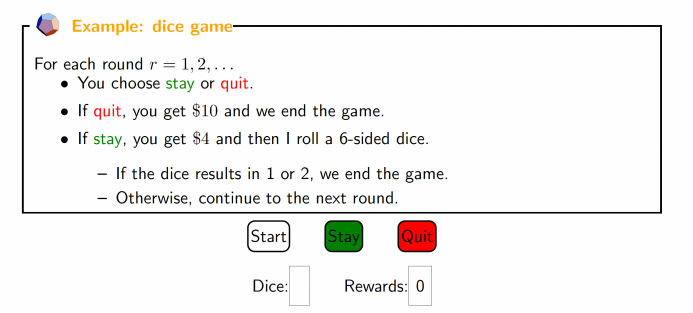



Figure – Source : Cours d'IA de Stanford

# Processus de décision de Markov

## Jeu de dés : récompense

Figure – Source : Cours d'IA de Stanford

Utilité attendue

$$\frac{1}{3}(4) + \frac{2}{3}.\frac{1}{3}(8) + \frac{2}{3}.\frac{2}{3}.\frac{1}{3}(12)... = 12$$

# Processus de décision de Markov

## Jeu de dés : récompense

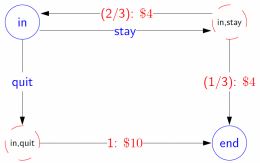
If follow policy "quit":

Figure – Source : Cours d'IA de Stanford

Utilité attendue

$$1(10) = 10$$

# Processus de décision de Markov : modélisation




Source : cours d'IA de Stanford

- États : situations possibles du système *ex. : score courant du joueur*
- Actions : choix possibles depuis un état *ex. : lancer le dé ou s'arrêter*
- Nœuds de probabilité : résultats aléatoires d'une action *ex. : valeur obtenue au dé*
- Transitions : résultats possibles menant à un nouvel état, associés à une probabilité et éventuellement à une récompense *ex. : nouveau score et gain associé*

# Processus de décision de Markov : modélisation

## Définition

- **États** : l'ensemble des états.
- **État initial** : $s_{\text{start}} \in \text{États}$.
- $\text{Actions}(s)$ : actions possibles à partir de l'état $s$.
- $T(s, a, s')$ : probabilité de $s'$ si l'action $a$ est prise à l'état $s$.
- $\text{Récompense}(s, a, s')$ : récompense pour la transition $(s, a, s')$.
- $\text{IsEnd}(s)$ : si la fin du jeu est atteinte.
- $0 \leq \gamma \leq 1$ : facteur d'escompte (par défaut : 1).

Les probabilités somment à un

- Pour chaque état $s$ et action $a$ :

$$\sum_{s' \in \text{États}} T(s, a, s') = 1$$

# Processus de décision de Markov : solution

- Dans les problèmes de recherche, une solution est un chemin (une séquence d'actions).
- Dans les MDP, une solution est une **politique.**

## Politique

Une politique $\pi$ est une correspondance de chaque état $s \in \text{États}$ à une action $a \in \text{Actions}$.

# Processus de décision de Markov : évaluation de la politique

## Utilité

- Suivre une politique produit un chemin aléatoire.
- L'utilité d'une politique est la somme (escomptée) des récompenses sur le chemin (c'est une variable aléatoire).

| Path | Utility |
|---|---|
| [in; stay, 4, end] | 4 |
| [in; stay, 4, in; stay, 4, in; stay, 4, end] | 12 |
| [in; stay, 4, in; stay, 4, end] | 8 |
| [in; stay, 4, in; stay, 4, in; stay, 4, in; stay, 4, end] | 16 |
| ... | ... |

Figure – Source : Cours d'IA de Stanford

La **valeur** d'une politique à un état est l'**utilité attendue**.

# Processus de décision de Markov : escompte

L'escompte capture le fait qu'une récompense aujourd'hui pourrait valoir plus que la même récompense demain.

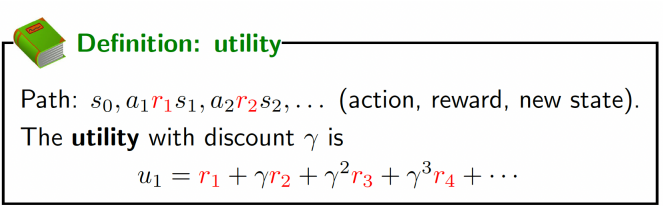



Figure – Source : Cours d'IA de Stanford

# Processus de décision de Markov : évaluation de politique

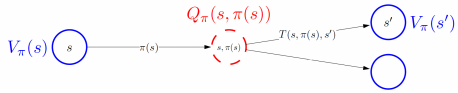



Figure – Source : Cours d'IA de Stanford

## Valeur d'une politique

$V_\pi(s)$ : utilité attendue obtenue en suivant la politique $\pi$ à partir de l'état $s$.

## Valeur Q d'une politique

$Q_\pi(s, a)$ : utilité attendue de prendre l'action $a$ à partir de l'état $s$, puis de suivre la politique $\pi$.

# Processus de décision de Markov : évaluation de politique

Définir des récurrences liant la valeur et la valeur Q

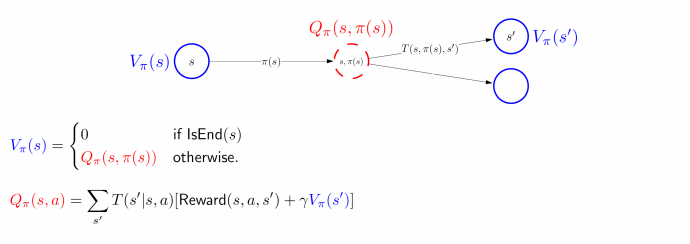
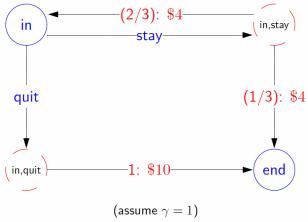



Figure – Source : Cours d'IA de Stanford

On obtient $Q_\pi(s, a)$ en considérant toutes les transitions possibles vers les états successeurs $s'$ et en prenant l'espérance sur la récompense immédiate $\text{Récompense}(s, a,')$ plus la récompense future escomptée $V_\pi(s')$.

# Processus de décision de Markov : évaluation de politique




Let $\pi$ be the "stay" policy: $\pi(\text{in}) = \text{stay}$.

$$V_\pi(\text{end}) = 0$$

$$V_\pi(\text{in}) = \tfrac{1}{3}(4 + V_\pi(\text{end})) + \tfrac{2}{3}(4 + V_\pi(\text{in}))$$

In this case, can solve in closed form:

$$V_\pi(\text{in}) = 12$$

Figure – Source : Cours d'IA de Stanford

# Processus de décision de Markov : évaluation de politique

**Key idea: iterative algorithm**

Start with arbitrary policy values and repeatedly apply recurrences to converge to true values.

**Algorithm: policy evaluation**

Initialize $V_\pi^{(0)}(s) \leftarrow 0$ for all states $s$.

For iteration $t = 1, \ldots, t_{\text{PE}}$:

For each state $s$:

$$V_\pi^{(t)}(s) \leftarrow \underbrace{\sum_{s'} T(s'|s, \pi(s))[\text{Reward}(s, \pi(s), s') + \gamma V_\pi^{(t-1)}(s')]}_{Q^{(t-1)}(s,\pi(s))}$$

# Exemple : DeepExploit

Framework de pentest automatisé utilisant l'apprentissage par renforcement pour rechercher des actions d'exploitation.

- État : OS, services, versions, ports ouverts de la cible
- Action : choix du payload ou de l'exploit à exécuter (via Metasploit)
- Récompense : succès ou échec de l'exploit
- But : compromettre la cible
- Politique : apprise par renforcement (réseau A3C), sans données préparées à l'avance

# Modèles logiques

# Données - Information - Connaissances

Modèle hiérarchique de la connaissance

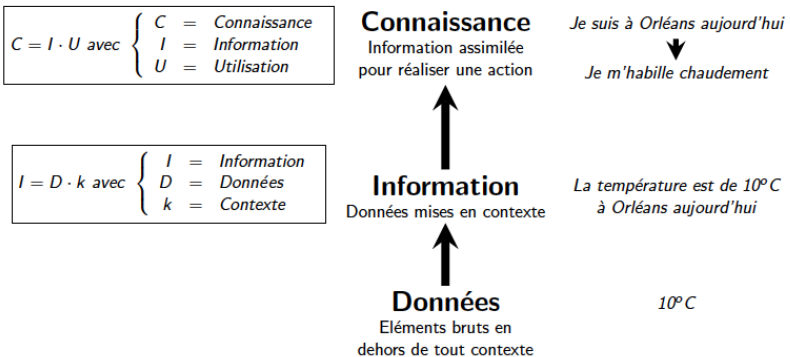



Figure – Source : T. Hamon

# Modèles logiques - Modèles à base de connaissance

## Agent fondé sur les connaissance

Agents qui utilisent un processus de raisonnement sur une représentation interne des connaissances pour décider des actions à entreprendre.

- **Représentation explicite** de la connaissance sur le monde ou le domaine d'application.
- **Raisonnement** à partir de cette connaissance : déduction, abduction,....

- Tirer parti de connaissances grâce à une capacité à combiner et recombiner des informations pour les adapter à une multitude de fins.
- Environnements partiellement observables : combiner connaissances générales et percepts reçus pour inférer des aspects cachés de l'état courant.
- Modèles très expressifs.

# Agent qui résonne logiquement

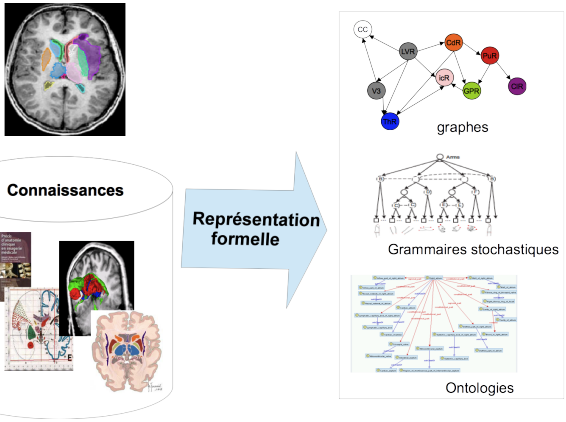
## Agent logique

Agent basé sur les connaissances disponibles concernant le monde et un raisonnement (logique) portant sur les actions dans ce monde.

Les agents logiques doivent connaître :

- L'état actuel du monde.
- Comment le monde change dans le temps ?
- Qu'est ce qu'il faut accomplir ?
- Quelles sont les conséquences des actions dans différentes circonstances ?

# de la Représentation de la connaissance…



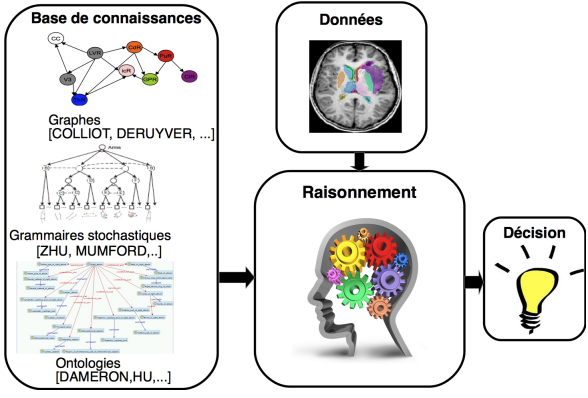
# Comment représenter les connaissances ?

## Les formalismes favorisant l'expression des connaissances (ontologie)

- Représentations structurées
  - Systèmes de frames
  - Réseaux sémantiques, Graphes conceptuels
  - Ontologies

- Représentations incertaines et imprécises
  - Réseaux bayésiens
  - Logique floue

# …au raisonnement pour une prise de décision



# Raisonnements

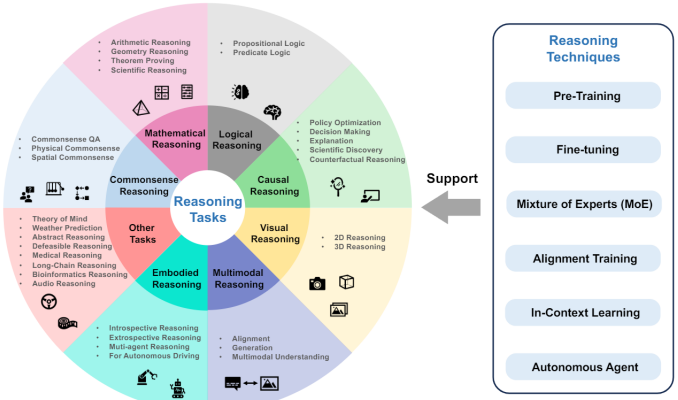
Comment utiliser (coordonner) les représentations pour en tirer des informations supplémentaires ?

## Types de raisonnements

- Abduction
- Déduction
- Induction
- Analogie
- Raisonnement par cas
- ...

# Une petite parenthèse sur le raisonnement

Modèles de fondation et raisonnement [1]




1. A Survey of Reasoning with Foundation Models, Sun et al : `https://arxiv.org/pdf/2312.11562.pdf`

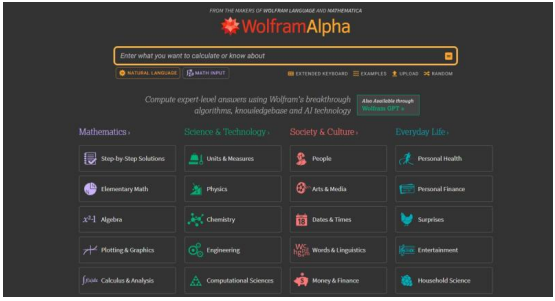
# Exemple 1 : Wolfram Alpha

## Un moteur de connaissance computationnelle

(Wolfram Research, 2009) répond à des questions en langage naturel en **calculant** la réponse à partir de bases de connaissances structurées, plutôt qu'en recherchant des documents.

## Capacités

- **Mathématiques** : équations, calcul symbolique
- **Sciences** : constantes, formules chimiques
- **Connaissances factuelles** : géographie, histoire, astronomie
- **Conversions** : unités, devises, calendriers

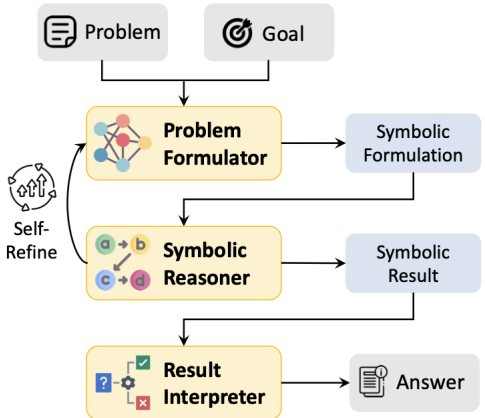


# Wolfram Alpha : un agent à base de connaissance

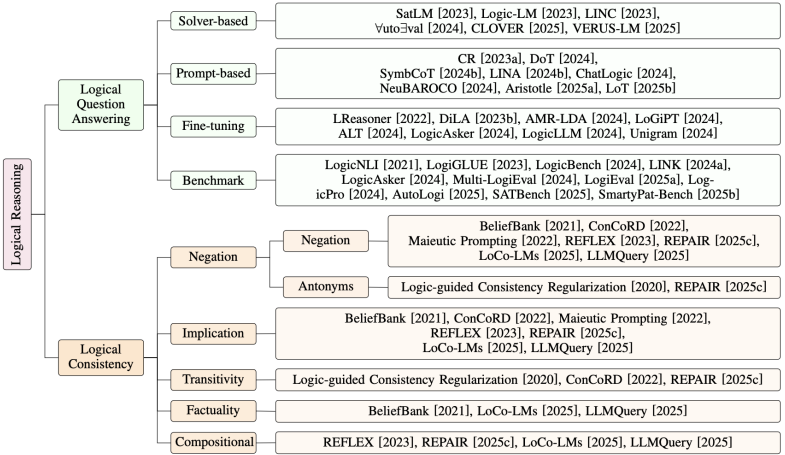
## Architecture

- Analyse de la requête en langage naturel $\rightarrow$ représentation structurée
- Base de connaissances *curée* (faits, formules)
- Moteur de calcul symbolique (issu de *Mathematica*)
- Réponse dérivée par **calcul**, non par recherche statistique

# Exemple 2 : Empowering Large Language Models with Symbolic Solvers



# Exemple : Empowering Large Language Models with logic reasoning




Source : https ://arxiv.org/pdf/2502.15652

# Résumé des types d'agents en une slide

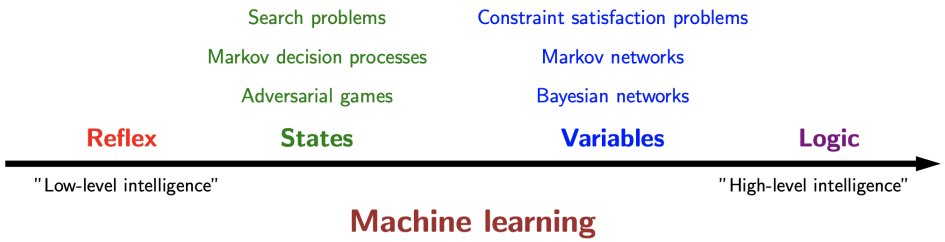
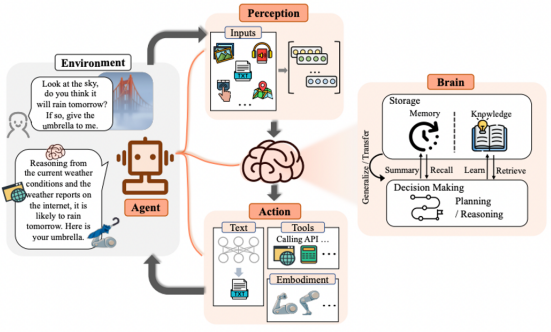



Figure – Source : cs221 course Stanford - Liang

# Et aujourd'hui, les agents à base de LLMs ou de modèles de fondation

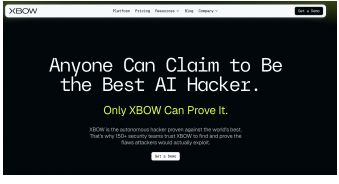



Orchestrer des pipelines complexes : applications LLM capables d'exécuter des tâches complexes grâce à l'utilisation d'une architecture qui combine les LLM avec des modules clés tels que la planification et la mémoire.

# XBOW : un agent autonome de pentest

XBOW est un agent autonome de test d'intrusion. En juin 2025, il est devenu le premier système autonome à atteindre la première place du classement HackerOne aux États-Unis, en découvrant et validant des vulnérabilités sur des cibles réelles.

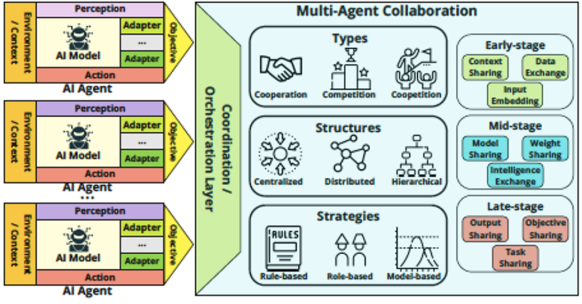
- Perception : collecte d'informations (scans, réponses HTTP, code source, etc.)
- Raisonnement : analyse des résultats et élaboration de pistes d'exploitation
- Action : exécute directement les outils et les exploits
- État : conserve un modèle du système exploré et met à jour les pistes à tester
- Adaptation : modifie sa stratégie en fonction des réponses obtenues, sans intervention humaine à chaque étape




Source : XBOW, 2025–2026

# IA agentique et IA incarnée (Embodied)

- **Coordination** and **orchestration** choices and architectures
- Communication structures and **topology**
  - centralized, distributed, flat, hierarchical,.
  - [Zhou el al, 25] Multi-agent system design : optimizing Agents with Better Prompts and Topologies
- **Small** Language Models or **Large** Language Models
  - [Belcak et al, 25] Small Language Models are the Future of Agentic AI

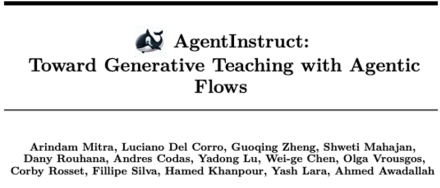



Source : [Tran et al, 25] Multi-agent Collaboration Mechanisms: A Survey of LLMs

# IA agentique et IA incarnée (Embodied)

## Pre and post-training agentic models

- Integrating agentic capabilities and tool-use in the training life-cycle
  - E.g. Pre-training : [Wu el al, 25] MaskSearch : A Universal Pre-Training Framework to Enhance Agentic Search Capability.
  - E.g. Post-training, often based on large-scale agentic data synthesis : [Kimi Team, 25] Kimi K2 : Open Agentic Intelligence

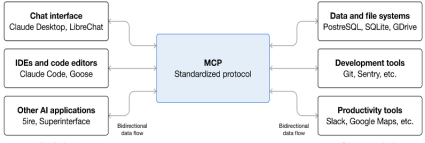


AgentInstruct:
Toward Generative Teaching with Agentic Flows

Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Codas, Yadong Lu, Wei-ge Chen, Olga Vrousgos, Corby Rosset, Fillipe Silva, Hamed Khanpour, Yash Lara, Ahmed Awadallah

Microsoft Research

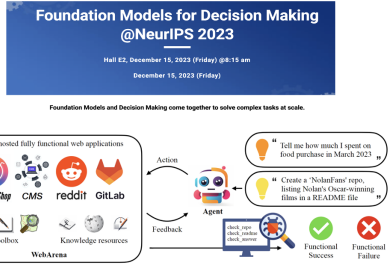
## Protocols and standards



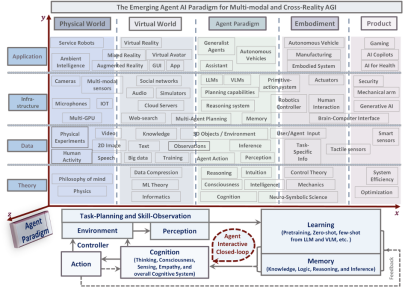

Model Context Protocol

# IA agentique et IA incarnée (Embodied)

## AI agent : A route towards Artificial General Intelligence (AGI)




Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents"




Durante et al., "Agent AI: Surveying the Horizons of Multimodal Interaction"

14

# Une petite parenthèse : l'AGI

## Définition

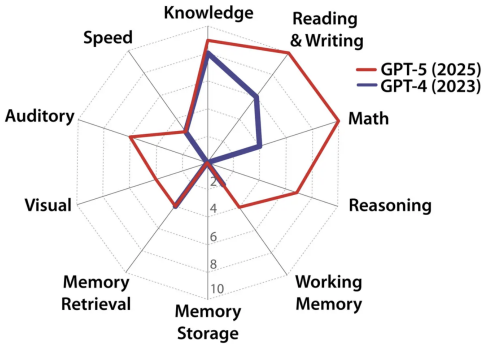
L'AGI est une IA capable d'égaler ou de dépasser la polyvalence et le niveau de compétence cognitive d'un adulte bien formé.

## Capacités

- Étude fondée sur la théorie des capacités cognitives de Cattell-Horn-Carroll (CHC), le modèle de l'intelligence humaine le plus largement validé sur le plan empirique
- Les systèmes d'IA contemporains sont capables de résoudre environ la moitié des évaluations, souvent simples, portant sur les capacités fondamentales qui sous-tendent la cognition humaine

Source : Hendrycks et al., A Definition of AGI, 2025 : `https://arxiv.org/abs/2510.18212` ou `https://www.agidefinition.ai/`

# Une petite parenthèse : l'AGI




The capabilities of GPT-4 and GPT-5.

Source : Hendrycks et al, A Definition of AGI, 2025 : https://arxiv.org/abs/2510.18212 or https://www.agidefinition.ai/

# Une petite parenthèse : l'AGI

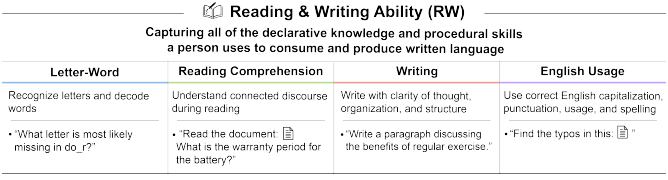
## 🎓 General Knowledge (K)

**Knowledge that is familiar to most well-educated people or is important enough that most adults have been exposed to it**

| Commonsense | Science | Social Science | History | Culture |
|---|---|---|---|---|
| Background knowledge about how the world works<br><br>• "What happens if you drop a glass bottle on concrete?"<br>• "Does making a sandwich take longer than baking bread?" | Knowledge of natural and physical sciences<br><br>**Physics**<br>• "A 2 kg object moves at constant velocity of 3 m/s. What is the net force?"<br><br>**Chemistry**<br>• "State the molecular geometry for the sulfur tetrafluoride molecule."<br><br>**Biology**<br>• "Which molecule is the final electron acceptor in the electron transport chain of cellular respiration?" | Understanding of human behavior, societies, and institutions<br><br>**Psychology**<br>• "Name the Big Five personality traits."<br><br>**Microeconomics**<br>• "Define a positive externality."<br><br>**Macroeconomics**<br>• "What's the difference between nominal and real interest rates?"<br><br>**Geography**<br>• "Which continent is the most threatened by desertification?"<br><br>**Comparative Government**<br>• "Describe the role of the Guardian Council in Iran." | Knowledge of past events and objects<br><br>**European History**<br>• "What were the main goals of the Congress of Vienna in 1815?"<br><br>**US History**<br>• "Analyze the goals of the Civil Rights Movement of the 1950s"<br><br>**World History**<br>• "Describe the end of the Cold War"<br><br>**Art History**<br>• "Discuss the use of contrapposto in ancient Greek and Roman sculpture" | Cultural literacy and awareness<br><br>**Current Affairs**<br>• "Who's the president of the United States?"<br><br>**Popular Culture**<br>• "Who is this?"<br> |

Source : Hendrycks et al, A Definition of AGI, 2025 : `https://arxiv.org/abs/2510.18212` or `https://www.agidefinition.ai/`

# Une petite parenthèse : l'AGI



**Reading & Writing Ability (RW)**

**Capturing all of the declarative knowledge and procedural skills a person uses to consume and produce written language**

| Letter-Word | Reading Comprehension | Writing | English Usage |
|---|---|---|---|
| Recognize letters and decode words | Understand connected discourse during reading | Write with clarity of thought, organization, and structure | Use correct English capitalization, punctuation, usage, and spelling |
| • "What letter is most likely missing in do_r?" | • "Read the document: What is the warranty period for the battery?" | • "Write a paragraph discussing the benefits of regular exercise." | • "Find the typos in this: " |

Source : Hendrycks et al, A Definition of AGI, 2025 : `https://arxiv.org/abs/2510.18212` or `https://www.agidefinition.ai/`

# Une petite parenthèse : l'AGI

**Mathematical Ability (M)**

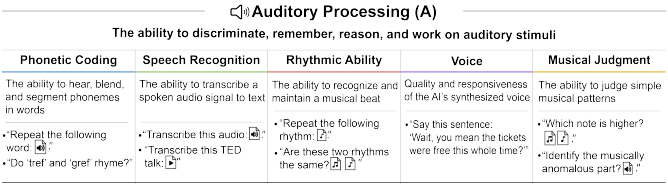
**The depth and breadth of mathematical knowledge and skills**

| Arithmetic | Algebra | Geometry |
|---|---|---|
| The manipulation of numbers using basic operations and solving word problems | The study of symbols and the rules for combining them to express general relationships and solve equations | The study of shapes, space, size, position, and distance |
| • Janet had 22 green pens and 10 yellow pens. She bought 6 bags of 9 blue pens and 2 bags of 6 red pens. How many pens does she have now? | • The first three terms of a geometric sequence are the integers $a, 720, b$, where $a < 720 < b$. What is the sum of the digits of the least possible value of $b$? | • An orange shaded rectangle is inscribed in a quarter-circle. Two sides of the rectangle lie along the two perpendicular radii of the quarter-circle, and the rectangle's edge touches the quarter-circle arc. Two segments are 2 and 4 units. What is the area of the orange shaded rectangle? |

| Probability | Calculus |
|---|---|
| The quantification of uncertainty by assigning numbers from 0 to 1 to events | The mathematics of change and accumulation |
| • Suppose $N$ dice are rolled, where $1 \leq N \leq 6$. Given that no two of the $N$ dice show the same face, what is the probability that one of the dice shows a six? Give a formula in terms of $N$. | • For what value of $k$ if any is $\int_0^\infty kxe^{-2x}\,dx = 1$ |

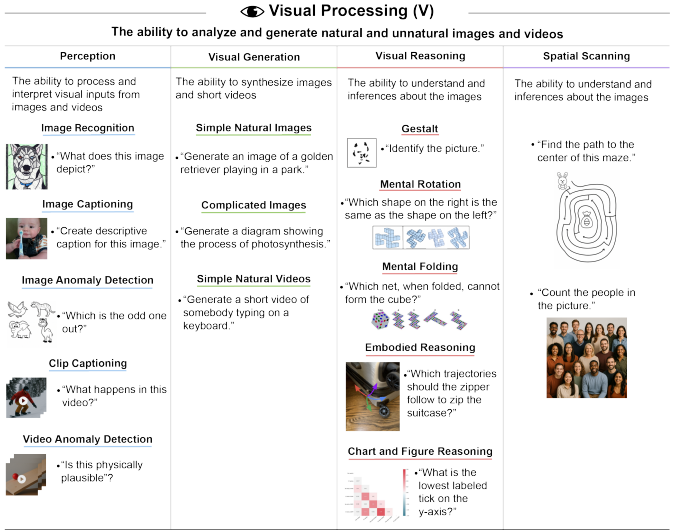
Source : Hendrycks et al, A Definition of AGI, 2025 : https://arxiv.org/abs/2510.18212 or https://www.agidefinition.ai/

# Une petite parenthèse : l'AGI

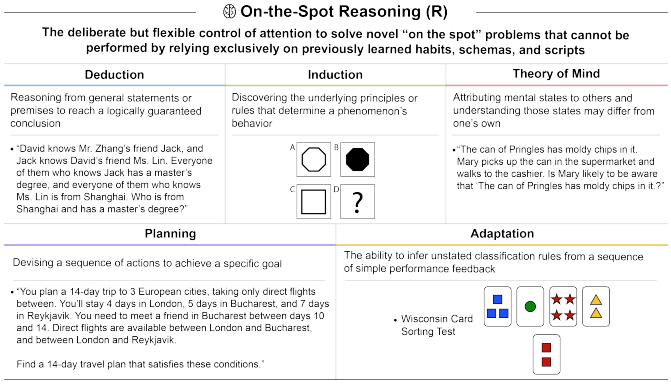



Source : Hendrycks et al, A Definition of AGI, 2025 : https://arxiv.org/abs/2510.18212 or https://www.agidefinition.ai/

# Une petite parenthèse : l'AGI

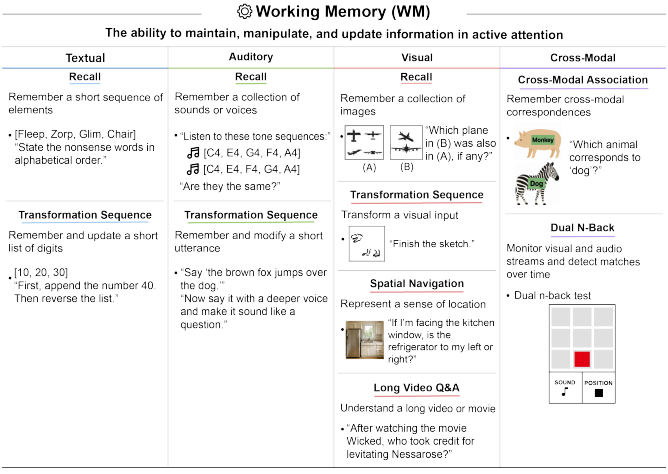


# Une petite parenthèse : l'AGI



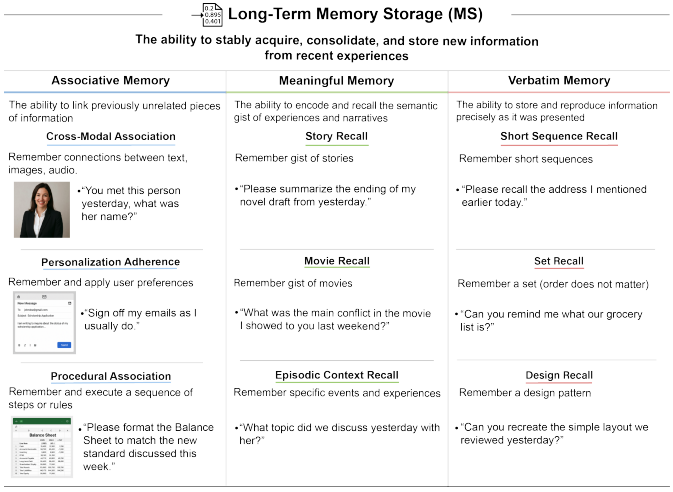

Source : Hendrycks et al, A Definition of AGI, 2025 : https://arxiv.org/abs/2510.18212 or https://www.agidefinition.ai/

# Une petite parenthèse : l'AGI



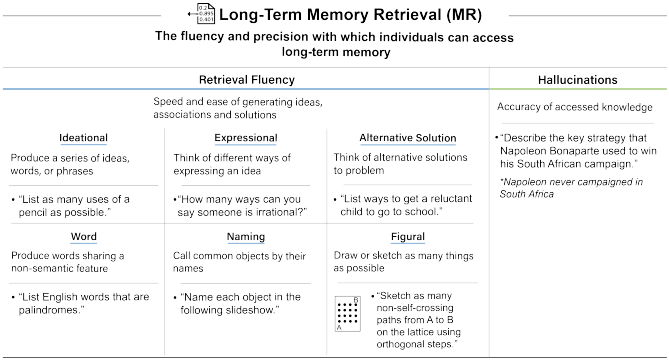

Source : Hendrycks et al, A Definition of AGI, 2025 : https://arxiv.org/abs/2510.18212 or https://www.agidefinition.ai/

# Une petite parenthèse : l'AGI




Source : Hendrycks et al, A Definition of AGI, 2025 : https://arxiv.org/abs/2510.18212 or https://www.agidefinition.ai/

# Une petite parenthèse : l'AGI



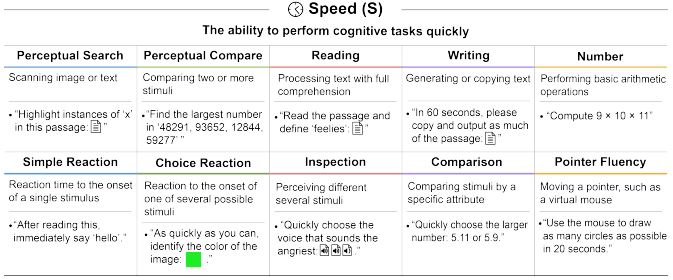
**Long-Term Memory Retrieval (MR)**

**The fluency and precision with which individuals can access long-term memory**

| Retrieval Fluency | | | Hallucinations |
|---|---|---|---|
| Speed and ease of generating ideas, associations and solutions | | | Accuracy of accessed knowledge |
| **Ideational**<br>Produce a series of ideas, words, or phrases<br>• "List as many uses of a pencil as possible." | **Expressional**<br>Think of different ways of expressing an idea<br>• "How many ways can you say someone is irrational?" | **Alternative Solution**<br>Think of alternative solutions to problem<br>• "List ways to get a reluctant child to go to school." | • "Describe the key strategy that Napoleon Bonaparte used to win his South African campaign."<br>**Napoleon never campaigned in South Africa* |
| **Word**<br>Produce words sharing a non-semantic feature<br>• "List English words that are palindromes." | **Naming**<br>Call common objects by their names<br>• "Name each object in the following slideshow." | **Figural**<br>Draw or sketch as many things as possible<br>• "Sketch as many non-self-crossing paths from A to B on the lattice using orthogonal steps." | |

Source : Hendrycks et al, A Definition of AGI, 2025 : https://arxiv.org/abs/2510.18212 or https://www.agidefinition.ai/

# Une petite parenthèse : l'AGI



**Speed (S)**

The ability to perform cognitive tasks quickly

| Perceptual Search | Perceptual Compare | Reading | Writing | Number |
|---|---|---|---|---|
| Scanning image or text | Comparing two or more stimuli | Processing text with full comprehension | Generating or copying text | Performing basic arithmetic operations |
| • "Highlight instances of 'x' in this passage: [doc]" | • "Find the largest number in '48291, 93652, 12844, 59277' " | • "Read the passage and define 'feelies': [doc]" | • "In 60 seconds, please copy and output as much of the passage: [doc]" | • "Compute 9 × 10 × 11" |
| **Simple Reaction** | **Choice Reaction** | **Inspection** | **Comparison** | **Pointer Fluency** |
| Reaction time to the onset of a single stimulus | Reaction to the onset of one of several possible stimuli | Perceiving different several stimuli | Comparing stimuli by a specific attribute | Moving a pointer, such as a virtual mouse |
| • "After reading this, immediately say 'hello'." | • "As quickly as you can, identify the color of the image: [■] ." | • "Quickly choose the voice that sounds the angriest: [🔊][🔊][🔊] " | • "Quickly choose the larger number: 5.11 or 5.9." | • "Use the mouse to draw as many circles as possible in 20 seconds." |

Source : Hendrycks et al, A Definition of AGI, 2025 : `https://arxiv.org/abs/2510.18212` or `https://www.agidefinition.ai/`

# Agents pour la cybersécurité

# Pour résumer

| Type d'agent | État interne | Planification | Utilité | Apprentissage | Architecture typique |
|---|---|---|---|---|---|
| Agent réactif | ✗ | ✗ | ✗ | ✗ | Règles condition–action |
| Agent cognitif | ✓ | ✓ | Optionnel | ✗ | BDI, raisonnement symbolique |
| Agent rationnel | ✓ | ✓ | ✓ | Optionnel | Modèle décisionnel |
| Agent apprenant | Variable* | Variable | Variable | ✓ | Apprentissage par renforcement, apprentissage supervisé |
| Agent LLM | ✓(mémoire textuelle) | ✓(basée sur les prompts) | Optionnel | Optionnel | LLM + outils + mémoire + planificateur |

Les agents LLM constituent une architecture flexible pouvant présenter des comportements réactifs, cognitifs, rationnels ou apprenants selon la conception du système.

* Cela dépend de la conception du système.

# Séance 29/09 : Prendre en main et utiliser de façon critique un environnement de développement augmenté par l'IA

# Objectifs de la séance

## Objectif principal

Mettre en place un environnement de développement augmenté par une IA générative et l'utiliser de manière critique dans une tâche de développement logiciel.

- Configurer un environnement de développement augmenté par une IA générative intégrée à l'IDE
- Décomposer une tâche de développement pour l'exploiter efficacement avec un assistant IA
- Évaluer de façon critique la pertinence, la fiabilité et les limites des suggestions produites
- Documenter et justifier ses choix d'usage de l'IA de façon traçable
- Discuter les enjeux de souveraineté, sécurité, propriété du code et dépendance à l'outil

# Une compétence, pas un outil

## Constat

En entreprise, l'assistance par IA générative est déjà la norme.

## Ce qui fait la différence

La compétence n'est plus savoir utiliser un assistant IA, mais savoir quand lui faire confiance, quand le vérifier, quand s'en passer, et comment rester responsable du résultat produit.

# Règles du jeu

- L'usage de l'IA est encouragé et attendu – l'objectif est de l'utiliser intensivement, pas de s'en passer
- Toute suggestion acceptée doit être relue et comprise avant intégration : vous restez responsables de tout le code rendu
- Toute modification issue de l'IA doit être testée avant d'être considérée comme acquise
- Vous documentez en continu un journal de prompts, avec au moins deux cas où l'IA s'est trompée ou a halluciné

# Déroulé de la séance

## Exercice 1
Mini-projet jouet : diagnostiquer un bug, écrire une fonction à partir d'une spec, écrire des tests, tester l'annulation, comparer deux modèles.

## Exercice 2
Même logique, avec un flux git complet : cloner, lire l'historique, traiter deux issues.

## Exercice 3
Discussion en binôme : comment ces réflexes changent d'échelle sur un vrai dépôt.

## Mission
Reprendre une pull request déjà fusionnée sur un dépôt réel, repartir du commit précédent, et refaire le travail avec l'IA.

# La mission

- Par binôme, sur un dépôt de votre choix (liste indicative fournie par mention)
- On repart du commit immédiatement antérieur à une pull request déjà fusionnée
- La PR fusionnée sert de version de référence pour votre analyse
- Point d'attention : sur des dépôts très connus, l'IA peut mémoriser le correctif plutôt que raisonner sur le code

# Livrables attendus

## Le code
Commité avec des messages clairs, dépôt GitHub partagé.

## Journal de prompts
Rempli en continu (EDUNAO) : demande, réponse, décision et justification.

## Fiche d’analyse critique
Remplie en séance (EDUNAO).

## Note de prise de recul
1 à 2 pages, à rendre sous une semaine : gains, pertes, limites, enjeux.

# Les questions qui comptent vraiment

- Qu'est-ce que l'IA vous a fait gagner ou perdre – vraiment ?
- Auriez-vous détecté les erreurs sans tests ni revue de code ?
- Qui est responsable si un bug généré par l'IA part en production ?
- Que faudrait-il mettre en place dans une équipe pour encadrer sainement cet usage ?